

Sparse Dictionary Learning for Single-Cell Classification

Theory and Design Note

Jonathan Ma
EN.553.740 Machine Learning 1
Instructor: Dr. Sichen Yang

October 7, 2026

1 Purpose and scope

This note specifies the dictionary-learning component of *Interpretable Gene Programs or Foundation Models? Comparing Sparse Dictionary Learning and scGPT for Single-Cell Classification*. The goal is to learn an inspectable representation from the target training data and compare its classification utility with frozen scGPT embeddings and a processed-expression baseline. This is a proposed design, not a report of completed experiments. The initial implementation will establish a reproducible fit, transform, and export interface before large-scale tuning.

2 Representation and objective

Let $X \in \mathbb{R}^{n \times p}$ be the processed training expression matrix, with cells in rows and retained genes in columns. Let $D \in \mathbb{R}^{K \times p}$ be a dictionary whose rows d_k are gene programs, and let $Z \in \mathbb{R}^{n \times K}$ contain cell-specific coefficients. We approximate

$$X \approx ZD, \quad x_i \approx \sum_{k=1}^K z_{ik} d_k.$$

The proposed estimator solves

$$\min_{Z,D} \frac{1}{2} \|X - ZD\|_F^2 + \lambda \|Z\|_1 \quad \text{subject to} \quad \|d_k\|_2 \leq 1 \quad (k = 1, \dots, K), \quad (1)$$

where $\|Z\|_1 = \sum_{i,k} |z_{ik}|$ and $\lambda > 0$. Reconstruction encourages fidelity to expression structure, while the entrywise ℓ_1 penalty encourages sparse cell representations.

The dictionary constraint prevents a scaling degeneracy: replacing (Z, D) by $(Z/c, cD)$ leaves ZD unchanged but divides the penalty by c . Bounding atom norms prevents this escape. Signed coefficients and signed atoms are allowed in the initial design, consistent with Equation (1); non-negative factorization would be a different model and is not assumed.

The numerical meaning of λ depends on the preprocessing scale and objective convention. The implementation must document whether reconstruction error is summed or averaged and match regularization accordingly. Dividing the *entire* objective by n preserves its minimizers; dividing only the reconstruction term does not preserve the same effective penalty.

3 Sparse coding and optimization

3.1 Encoding with a fixed dictionary

For a fixed D , rows of Z can be estimated independently:

$$\hat{z}_i = \arg \min_{z \in \mathbb{R}^K} \left\{ \frac{1}{2} \|x_i - zD\|_2^2 + \lambda \|z\|_1 \right\}. \quad (2)$$

This is a convex Lasso problem [1] with design matrix $D^\top$; correlated or redundant atoms can make the solution nonunique. Larger penalties favor smaller coefficient magnitudes and, typically, fewer active atoms, but predictive accuracy need not change monotonically.

For illustration, a proximal-gradient update is

$$z^{(t+1)} = S_{\eta\lambda} \left(z^{(t)} - \eta(z^{(t)}D - x_i)D^\top \right),$$

where $S_a(u) = \text{sign}(u) \max(|u| - a, 0)$ acts entrywise. For nonzero D , a step size $0 < \eta \leq 1/\|DD^\top\|_2$ is a standard conservative choice. Proximal methods and their accelerated variants are developed by Beck and Teboulle [4]. The update above is the unaccelerated step, not the full FISTA algorithm. This explains the role of soft thresholding; it does not require writing a custom solver for the first baseline.

3.2 Learning the dictionary

With Z fixed, updating D is a convex constrained least-squares problem. A projected-gradient step takes

$$\tilde{D} = D - \eta_D Z^\top (ZD - X), \quad d_k^{\text{new}} = \frac{\tilde{d}_k}{\max(1, \|\tilde{d}_k\|_2)}.$$

An appropriate step size is needed; arbitrary gradient steps followed by projection do not guarantee improvement. Alternating sparse-code and dictionary updates exploits the blockwise convex structure, but the joint problem is nonconvex and initialization matters. Exact block minimization cannot increase the objective, yet this does not establish convergence to a global optimum. Mini-batch or approximate updates require their own convergence diagnostics.

Mairal et al. [3] provide a scalable online approach to dictionary learning with sparsity penalties and constrained atoms. The first implementation should use an established dictionary-learning solver, with its loss convention, encoding algorithm, stopping tolerances, and random seed recorded. In particular, verify that the transformation routine solves the intended penalized sparse-coding problem rather than silently using a different sparsity rule. Monitor reconstruction, sparsity, iteration limits, and unused atoms.

4 Preprocessing and leakage boundaries

The preprocessing teammate should provide stable, unique cell IDs; labels; shared training, validation, and test assignments; ordered gene IDs; and available donor, study, or sequencing-technology metadata. Choose grouped splits when scientifically appropriate metadata permit them. Technology is not automatically a donor identifier.

The proposed classical pipeline uses documented normalized/log-transformed expression, training-selected variable genes, and a documented training-fitted scaling rule. Exact choices

remain to be agreed with preprocessing. Do not assume a layer named `counts` establishes raw-count provenance. Preserve the original data and record transformations explicitly.

Fit gene selection, centering/scaling, and dictionary estimation on training cells only. Apply the fitted gene ordering and transforms unchanged to validation and test cells. For cross-validation, fit these components separately within each training fold. Avoid densifying the full expression matrix unnecessarily; centering sparse matrices can greatly increase memory use.

scGPT may require its own fixed panel and raw-count processing. The dictionary gene set need not match that panel. The comparison must share cell identities, labels, and evaluation splits, not force incompatible input transformations. Any scGPT-related cell exclusion must be resolved centrally so all methods are evaluated on the same retained cells.

5 Training, tuning, and held-out encoding

1. Fit the classical preprocessing on the training partition and construct X_{train} .
2. Learn D and training codes for a small, prespecified set of dictionary sizes K and penalties λ ; choose numerical penalty values only after the feature scale is fixed.
3. Encode validation cells using Equation (2) with D fixed. Train the common regularized multinomial logistic classifier on training codes and evaluate on validation codes.
4. Select settings using a stated training/validation procedure and budget. Record any classifier-side scaling and fit it only on training codes. Coordinate classifier regularization budgets across the representation methods.
5. Freeze the selected pipeline and evaluate the final test partition once. If refitting on training plus validation data is chosen, refit all classical components on that union and re-encode its cells and test cells consistently; do not combine codes from different fitted dictionaries.

The initial baseline should use one documented dictionary seed. Later robustness runs may vary initialization using the same splits. Dictionary seeds and scGPT pretraining sampling seeds describe different sources of variation and should not be treated as equivalent replicates. Neither representation parameters nor checkpoints should be selected using final test performance.

6 Interpretability and evaluation

Report classification Macro-F1 as the primary metric, together with balanced accuracy and multi-class predictive log loss. Use the same held-out cells and consistent class/probability ordering for every method. The classifier and comparative-analysis teammate owns these downstream evaluations.

For the dictionary component, report reconstruction error, the fraction of coefficients whose magnitudes exceed a declared numerical tolerance, and atom usage. Inspect genes with the largest positive and negative weights separately and summarize signed activation by cell type. These summaries are descriptive associations, not evidence of causal biological mechanisms.

Atom signs and ordering are not identifiable: flipping both an atom and its coefficient column, or jointly permuting atoms and coefficient columns, leaves reconstruction unchanged. Compare programs across runs only after appropriate matching and sign alignment. Correlated atoms can also split or share expression patterns, so a single atom should not automatically be interpreted as a unique pathway.

7 Proposed artifact contract

Artifact	Required contents
Cell codes	Z of shape (n_{cells}, K) with ordered, unique cell IDs and a dictionary/run identifier.
Dictionary	D of shape (K, p) with ordered gene IDs and atom identifiers.
Fitted pipeline	Gene selection, scaling parameters, encoding settings, K , λ , solver version, seed, and convergence diagnostics.
Provenance	Dataset/version or checksum, split-manifest identity, preprocessing choices, exclusions, and training-cell identity.

Prefer numeric arrays and string IDs that can be read without object pickling for shared feature exports. Labels and splits should come from the common cell-ID table. Validate joins explicitly: reject duplicate IDs, missing or extra cells, nonfinite features, and inconsistent gene ordering. Do not align representations by row position alone. Different representation dimensions are expected; no adapter to scGPT’s 128 dimensions is required.

8 First implementation milestone

Begin with a small synthetic matrix generated from a known sparse factorization, optionally with noise. Check the fit/transform/export path, finite values, atom norm constraints, dimensions, ID preservation, and whether held-out encoding leaves the dictionary unchanged. Confirm that export/reload preserves the representation. Because factors are nonidentifiable, do not demand exact elementwise recovery of the generating dictionary.

Then run a modest real-data pilot after preprocessing is agreed. Compare reconstruction with a simple reference such as the zero reconstruction, report sparsity and runtime, and inspect a few atoms before expanding tuning. Low reconstruction error alone does not establish useful cell-type information or biological interpretability.

9 Decisions still open

The team must finalize the compatible downstream data source, classical normalization and scaling, gene selection, shared split policy, cell exclusions, hyperparameter budget, and artifact filenames. This note fixes the mathematical formulation and integration boundaries while leaving those empirical choices explicit. It does not require the scGPT module to finish before development of the dictionary-learning baseline.

10 Relevant literature and reading priorities

Sparse representations. Olshausen and Field [2] give a foundational example of learning sparse representations from natural images. Their work motivates learning reusable components with sparse activities, but it is not evidence that gene-expression atoms necessarily correspond to biological pathways. Tibshirani [1] supplies the statistical basis for ℓ_1 -penalized estimation and is the first reading for understanding the fixed-dictionary coding problem.

Optimization and implementation. Mairal et al. [3] are the most directly relevant algorithmic reference for this module: they develop online matrix factorization and sparse dictionary learning for large datasets. Their convergence results require stated assumptions and should not be transferred automatically to any mini-batch implementation. Beck and Teboulle [4] explain accelerated proximal optimization for convex composite problems; this supports the sparse-coding subproblem, not a global-optimality claim for jointly learning Z and D .

Interpretable single-cell programs. Kotliar et al. [5] use consensus nonnegative matrix factorization to identify expression programs associated with cell identity and activity. This provides a useful biological and stability-oriented comparison for program interpretation. Their nonnegative model differs from our signed, sparsity-penalized formulation; we should borrow interpretation questions, not assume identical factor semantics or recovery guarantees.

Foundation-model comparison. Cui et al. [6] introduce scGPT and establish the foundation-model context. Our teammate’s small custom-pretrained scGPT checkpoints must be described separately from the published model’s weights and reported results. DenAdel et al. [7], the reference paper supplied with this project, investigate pretraining size and diversity across single-cell models and tasks. Their results motivate evaluating representation utility against simpler baselines, but do not determine the outcome of our dictionary-learning comparison.

Suggested reading order. Start with Tibshirani for the coding objective and Mairal et al. for the dictionary algorithm. Then read Kotliar et al. for gene-program interpretation, and Cui et al. together with DenAdel et al. for the comparison design. Use Beck and Teboulle when studying solver behavior, and Olshausen and Field for historical context.

References

- [1] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. [doi:10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x).
- [2] B. A. Olshausen and D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381:607–609, 1996. [doi:10.1038/381607a0](https://doi.org/10.1038/381607a0).
- [3] J. Mairal, F. Bach, J. Ponce, and G. Sapiro. Online learning for matrix factorization and sparse coding. *Journal of Machine Learning Research*, 11:19–60, 2010. <https://www.jmlr.org/papers/v11/mairal10a.html>.
- [4] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. [doi:10.1137/080716542](https://doi.org/10.1137/080716542).
- [5] D. Kotliar et al. Identifying gene expression programs of cell-type identity and cellular activity with single-cell RNA-Seq. *eLife*, 8:e43803, 2019. [doi:10.7554/eLife.43803](https://doi.org/10.7554/eLife.43803).
- [6] H. Cui et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. [doi:10.1038/s41592-024-02201-0](https://doi.org/10.1038/s41592-024-02201-0).
- [7] A. DenAdel et al. Evaluating the role of pretraining dataset size and diversity on single-cell foundation model performance. *Nature Methods*, 2026. [doi:10.1038/s41592-026-03120-y](https://doi.org/10.1038/s41592-026-03120-y). Project copy: `docs/references/denAdel.pdf`.